

# Model Context Protocol (MCP) : L'USB-C de l'IA

De l'intégration artisanale à la conception de systèmes agentiques autonomes, robustes et à l'échelle.

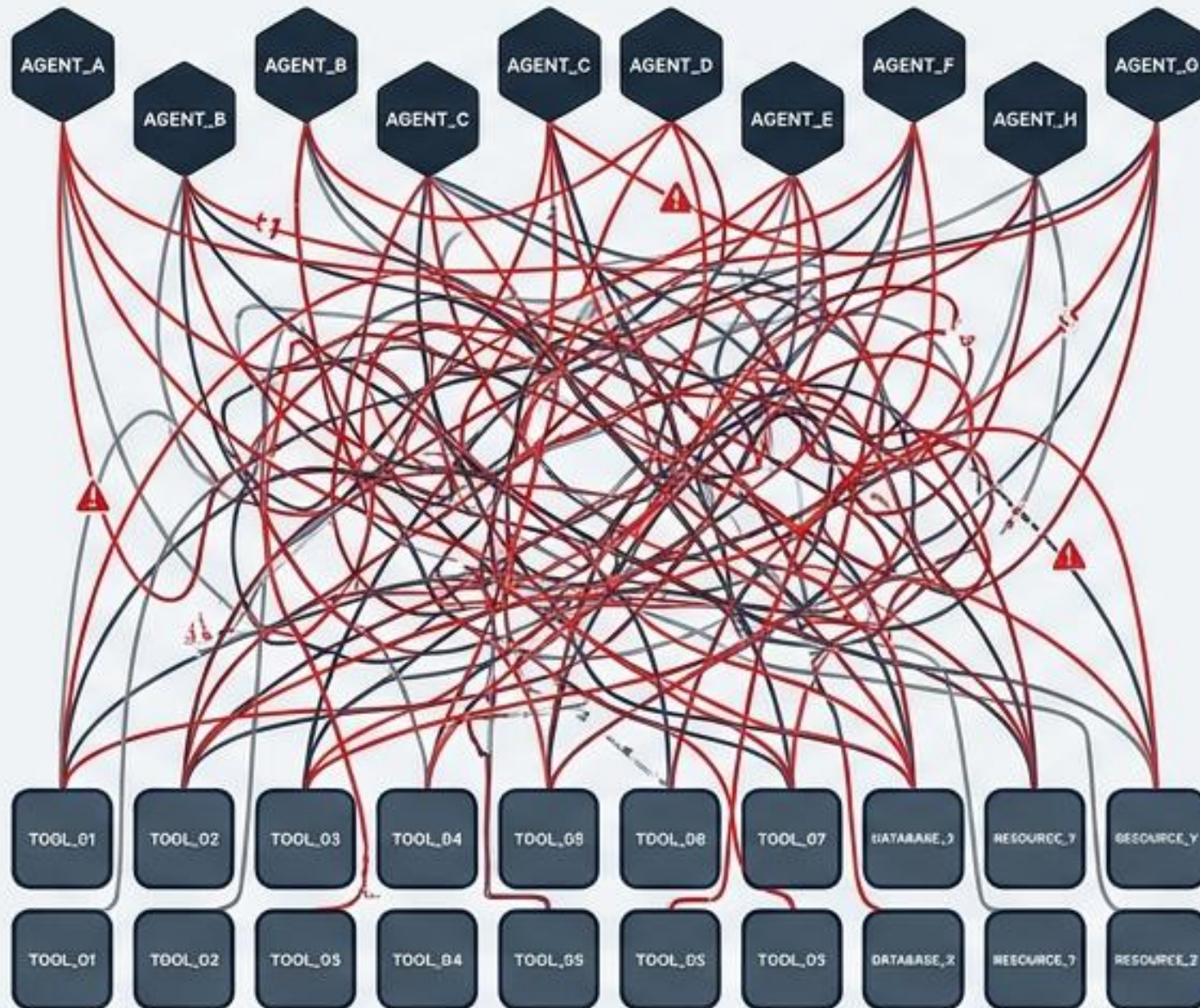


Guide Architectural & Enjeux de Production



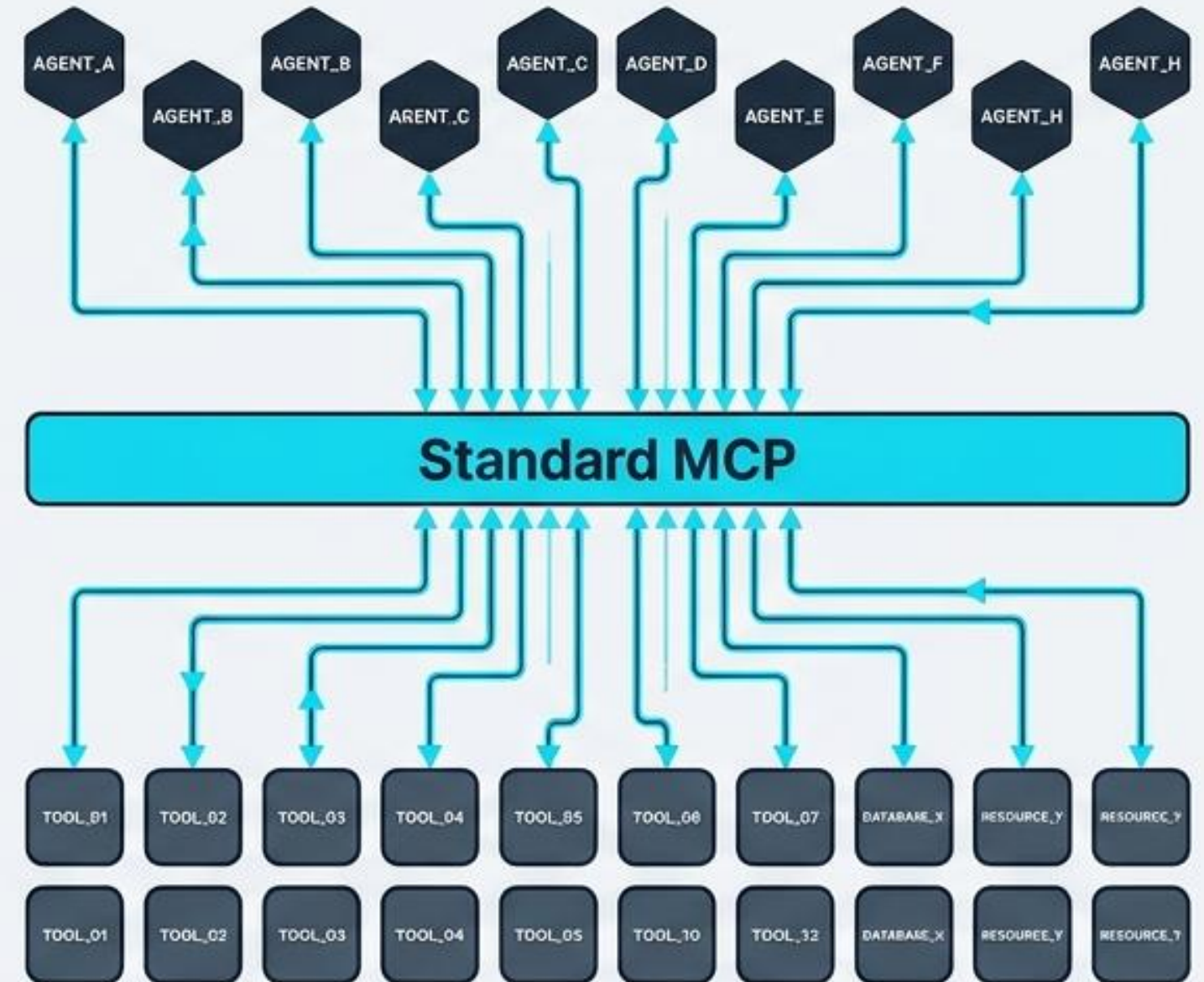
# Le Cauchemar de l'Intégration (N × M)

Le Cauchemar N × M



**Le Problème:** Chaque intégration IA exige un code sur mesure, des prompts spécifiques et une maintenance isolée.  
**L'Équation:** 10 Agents × 20 Outils = 200 connexions fragiles à maintenir.

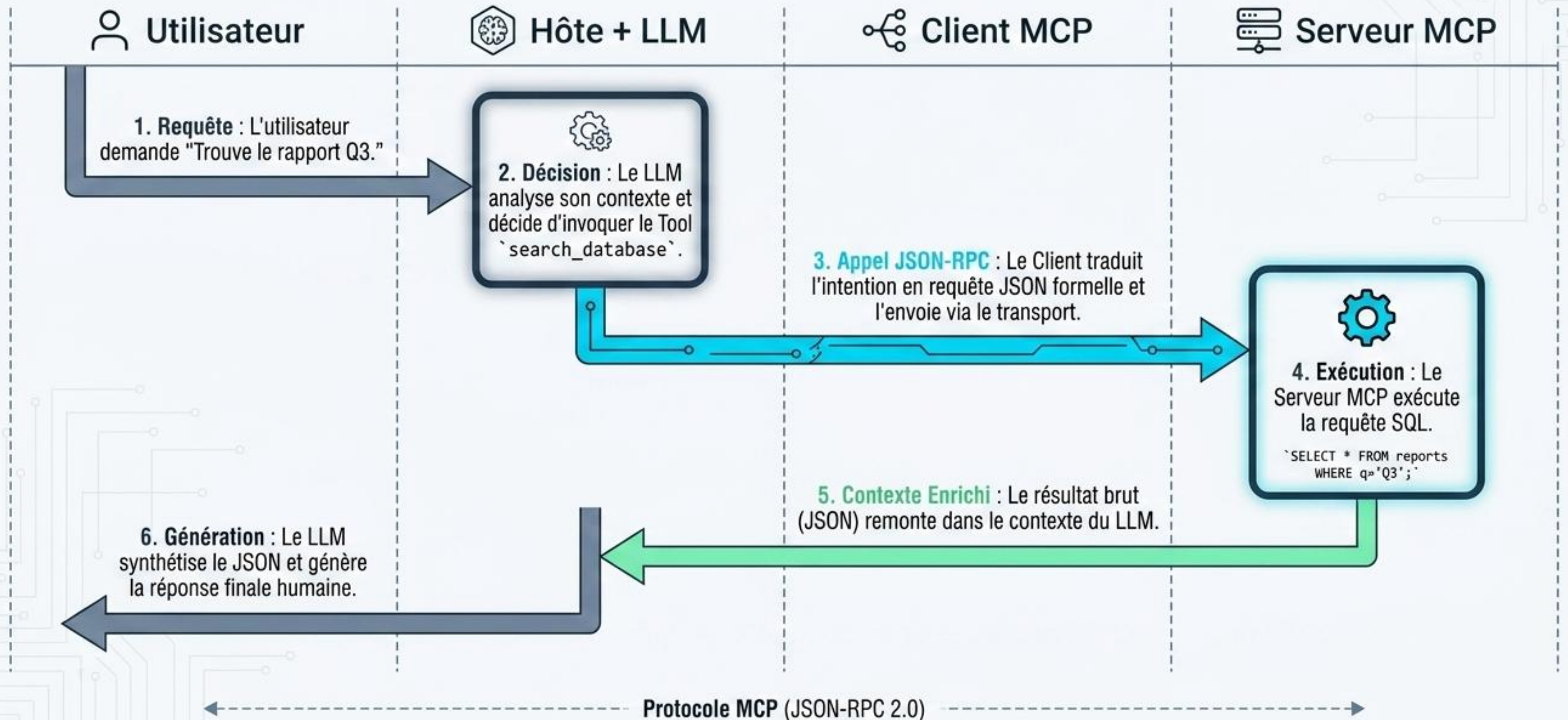
Le Paradigme N + M



**La Solution MCP:** Un contrat universel ( `JSON-RPC 2.0` ). Construisez le serveur une fois, tous les clients compatibles s'y connectent instantanément.

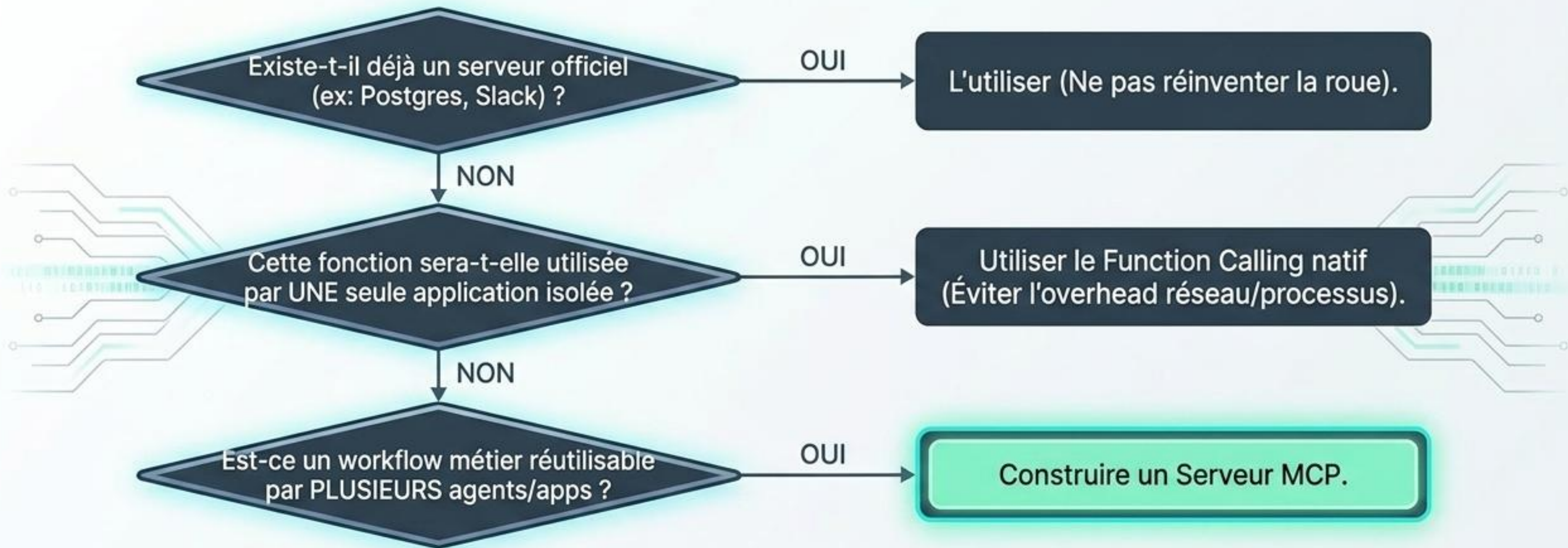


# Cycle de Vie d'une Requête MCP





# Arbre de Décision : Faut-il construire un Serveur MCP ?

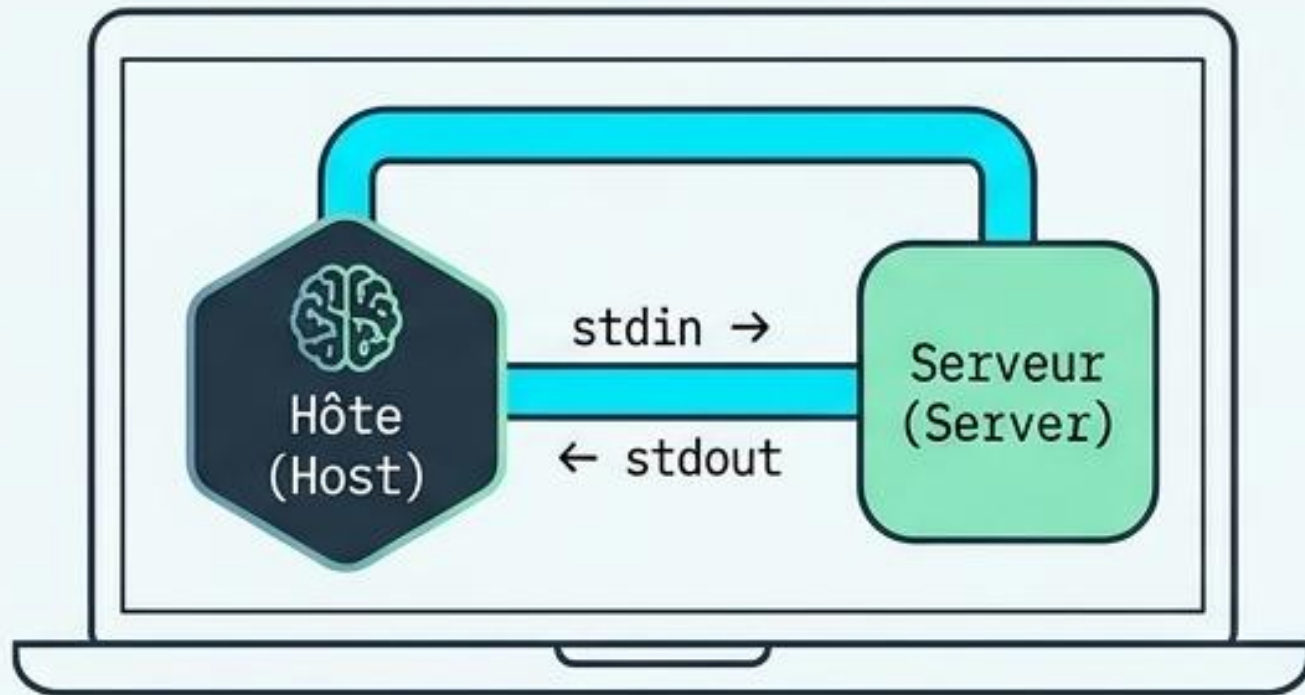


**Takeaway :** Le coût d'implémentation du protocole (maintenance, réseau) doit être rentabilisé par la réutilisabilité.



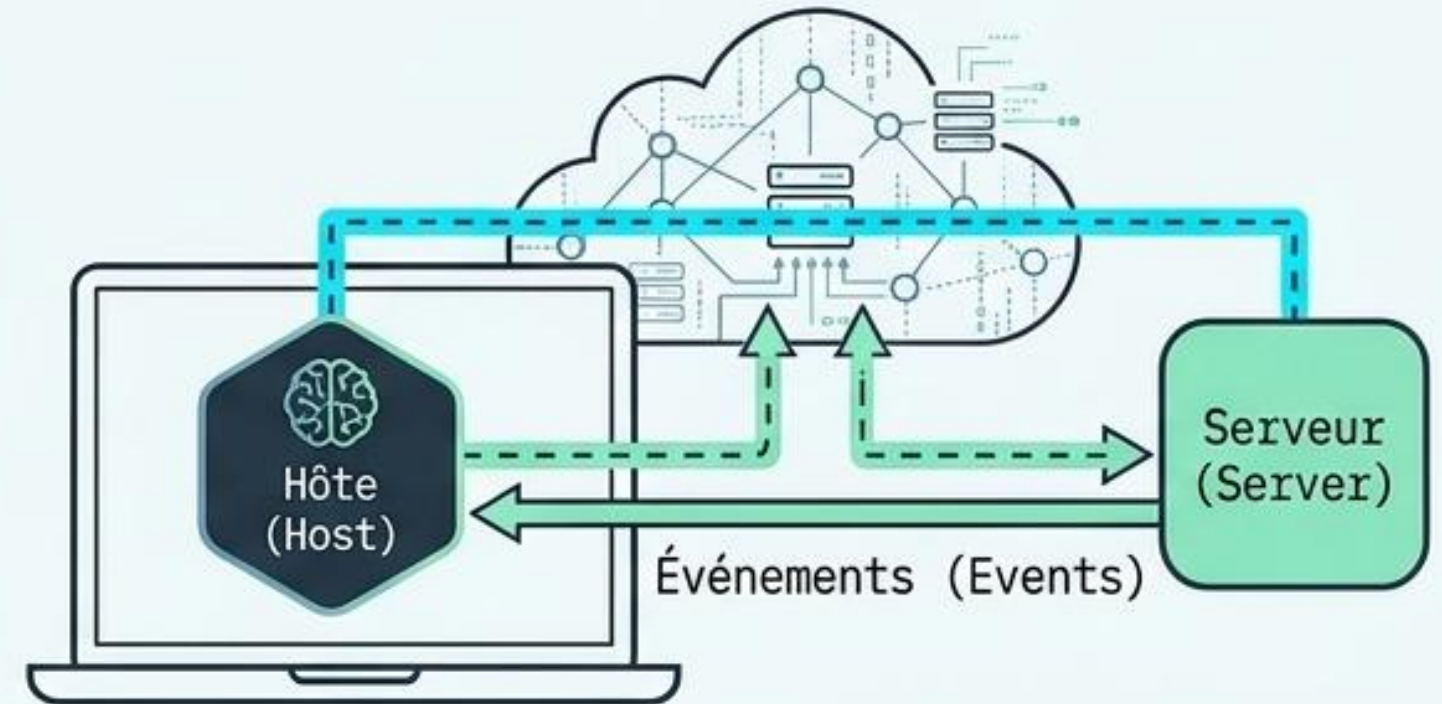
# La Mécanique de Transport : Agnostique & Universelle

## STDIO (Entrée/Sortie standard)



Le client lance le serveur comme sous-processus (stdin/stdout).  
Cas d'usage : Développement local, outils desktop, latence critique, sécurité air-gapped.

## SSE / HTTP (Server-Sent Events)



Communication réseau.  
Cas d'usage : Serveurs distants, SaaS, architecture multi-agents, production à l'échelle.

Le Handshake : À la connexion, client et serveur exécutent une Négociation des Capacités pour définir les primitives supportées.



# Les 3 Primitives Fondamentales



## Tools (Outils)

Action avec effet de bord  
(Écrire, Calculer, API POST).

Contrôlé par le **Modèle**.

Le LLM décide quand et  
comment l'appeler.

Vecteur de risque principal.



## Resources (Ressources)

Données en lecture seule,  
identifiées par URI (file://,  
postgres://).

Contrôlé par l'**Application**.

Injecte du contexte massif  
sans brûler des appels  
d'outils continus.



## Prompts (Templates)

Recettes de workflows  
pré-définies (ex: Analyse  
ce code).

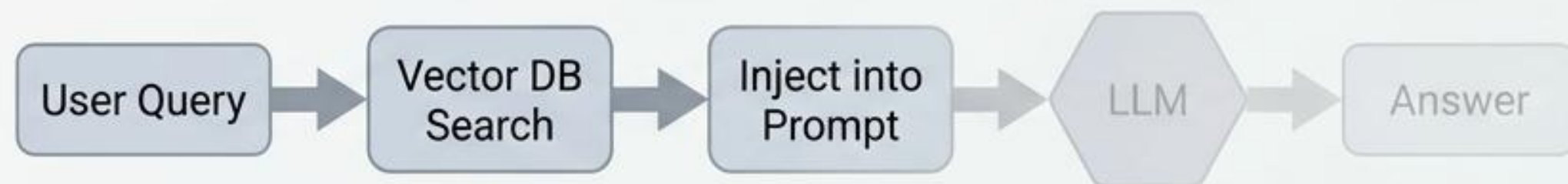
Contrôlé par l'**Utilisateur**.

Standardise les  
instructions complexes  
côté serveur.



# Cas d'Usage Phare : Le RAG Agentique

## RAG Linéaire (Approche Classique)



⚠ **La Limite du RAG Classique :**  
La recherche est une étape initiale fixe. Si le contexte récupéré est mauvais, **l'IA hallucine**.

## RAG Agentique via MCP (Approche Standardisée)



**La Solution MCP :** La base vectorielle devient un **Serveur MCP**.

**Le Résultat :** L'Agent LLM choisit quand chercher, quoi chercher, et peut raffiner itérativement ses **requêtes** avant de répondre à l'utilisateur.

L'API Google Calendar (Service Cloud)

Avant MCP, si vous vouliez que votre agent gère l'emploi du temps d'un utilisateur, vous deviez adapter l'interface de l'API Google à chaque "cerveau" :

| Rôle       | Nom technique |          | Responsabilité                                                           |
|------------|---------------|----------|--------------------------------------------------------------------------|
| L'Hôte     | MCP Host      | Le Cadre | Fournit l'interface utilisateur et le LLM (le cerveau).                  |
| Le Client  | MCP Client    | Le Pont  | Gère la connexion, demande la liste des outils, transmet les appels.     |
| Le Serveur | MCP Server    | L'Expert | Détient l'accès aux données (SQL, Fichiers, API) et exécute les actions. |

|                         |                                                                                                                                                                                              |
|-------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Pour OpenAI (ChatGPT)   | Il fallait créer un "Custom GPT Action" avec un fichier JSON respectant strictement la spécification OpenAPI, + configurer l'authentification <b>OAuth</b> dans leur interface propriétaire. |
| Pour Claude (Anthropic) | Il fallait définir le schéma de l'outil directement dans le code via leur SDK, avec une structure de blocs XML ou JSON légèrement différente.                                                |
| Problème                | Si vous vouliez que l'agent puisse aussi être utilisé dans une application mobile autonome ou un autre LLM, vous deviez maintenir plusieurs définitions de la fonction create_event().       |

 Modes de transport disponibles

studio

Subprocess local

⚡ Rapide, simple

✔ Utilisé dans le TP

SSE

Server-Sent Events

🌐 HTTP streaming

Pour serveurs web

HTTP

Streamable HTTP

☁️ Distribué

Production, cloud



# Matrice de Synthèse : Où se situe MCP ?

|                    | Function Calling (Natif)               | OpenAPI (REST)                      | Model Context Protocol                            |
|--------------------|----------------------------------------|-------------------------------------|---------------------------------------------------|
| Couche d'opération | Modèle IA                              | Spécification Web                   | <b>Intégration Agentique</b>                      |
| Portée             | Verrouillé au Fournisseur              | Agnostique (HTTP)                   | <b>Standard ouvert Multi-Modèles</b>              |
| Couvre l'UX LLM ?  | Non                                    | Non                                 | <b>Oui (Gère découverte, prompts, contexte)</b>   |
| Réutilisabilité    | Code métier à réécrire par application | Décrit les données, pas l'action IA | <b>1 Serveur = Compatible avec tous les hôtes</b> |

Takeaway : MCP n'est pas un concurrent du Function Calling. Il l'encapsule. C'est la couche réseau intermédiaire qui unifie les appels natifs sous le capot.



# Le Piège de la Démo : Les Limites en Production



## Ce que MCP fait parfaitement :

Standardiser la communication (Le Protocole).

## Ce que MCP NE FAIT PAS :

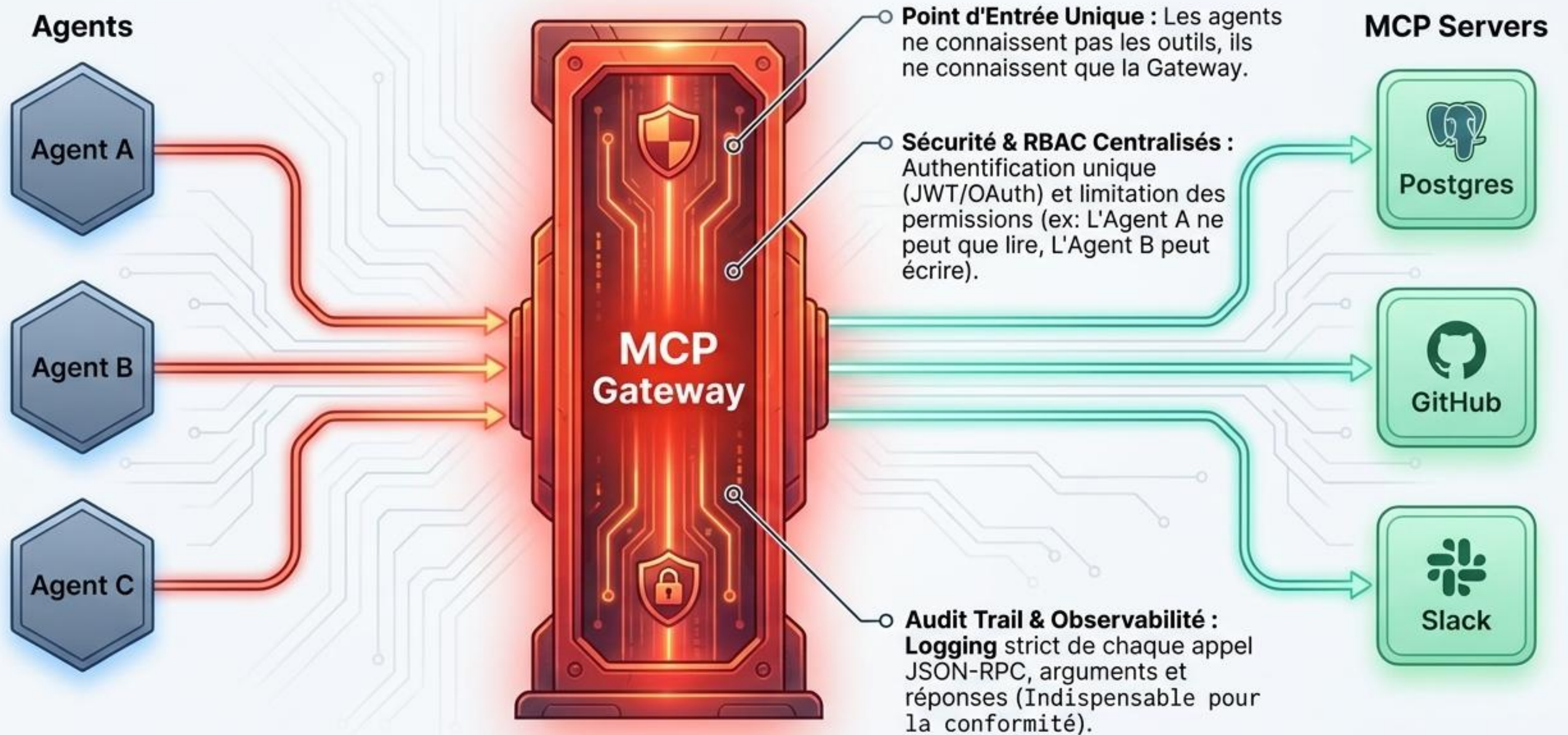
- **Authentification à l'échelle** : Qui possède quelles clés ?
- **Contrôle d'accès (RBAC)** : Quel agent a le droit d'utiliser quel outil ?
- **Observabilité** : Que s'est-il passé exactement ? (Audit logs).
- **Gouvernance** : Comment bloquer une injection de prompt (Prompt Injection) ?



**Le Verdict** : Laisser des agents accéder directement aux outils métiers via MCP sans couche de contrôle intermédiaire est un **risque de sécurité majeur**.

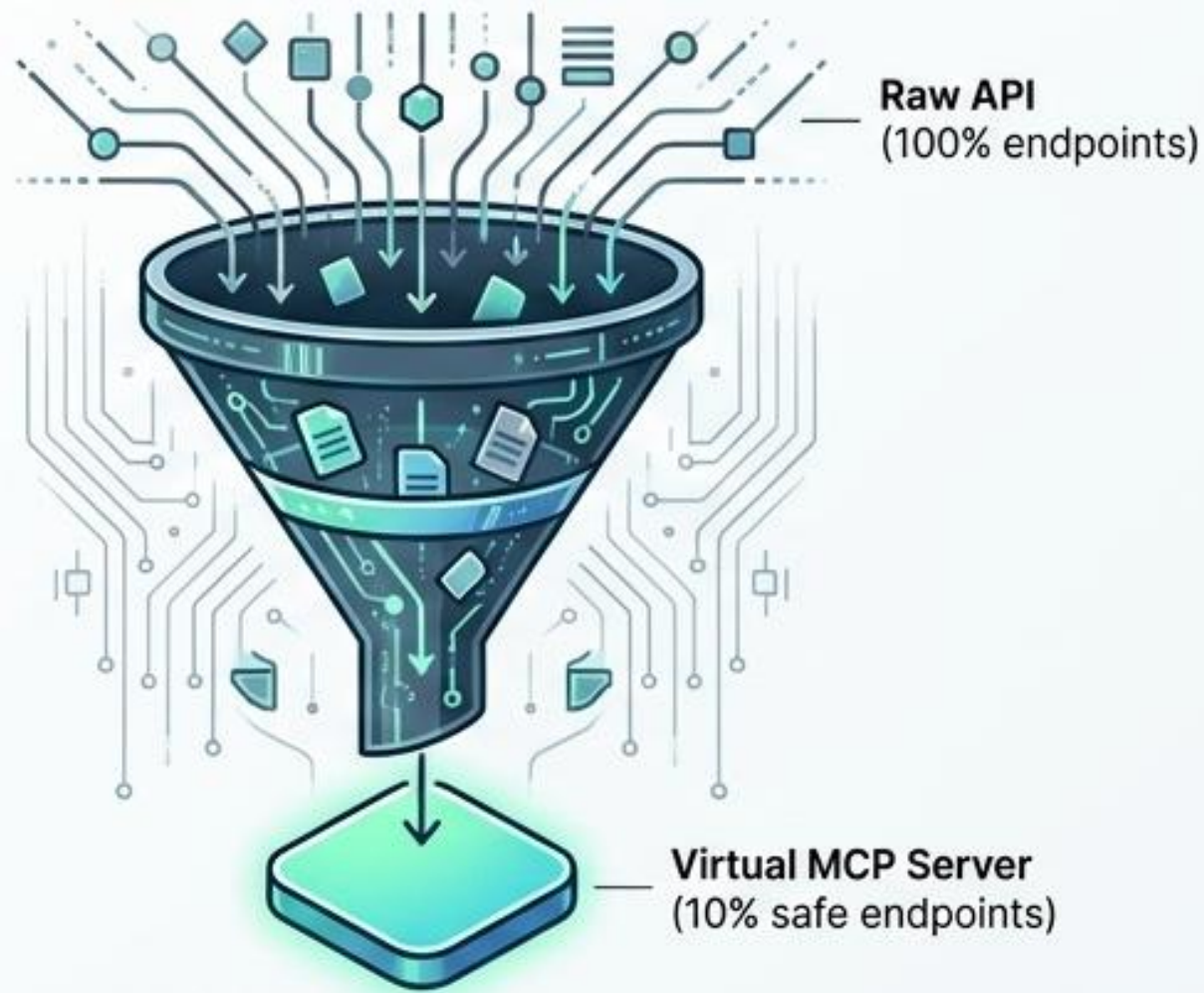


# L'Impératif Architectural : Le MCP Gateway





# Sécurité Avancée : Virtual MCP & Human-in-the-Loop



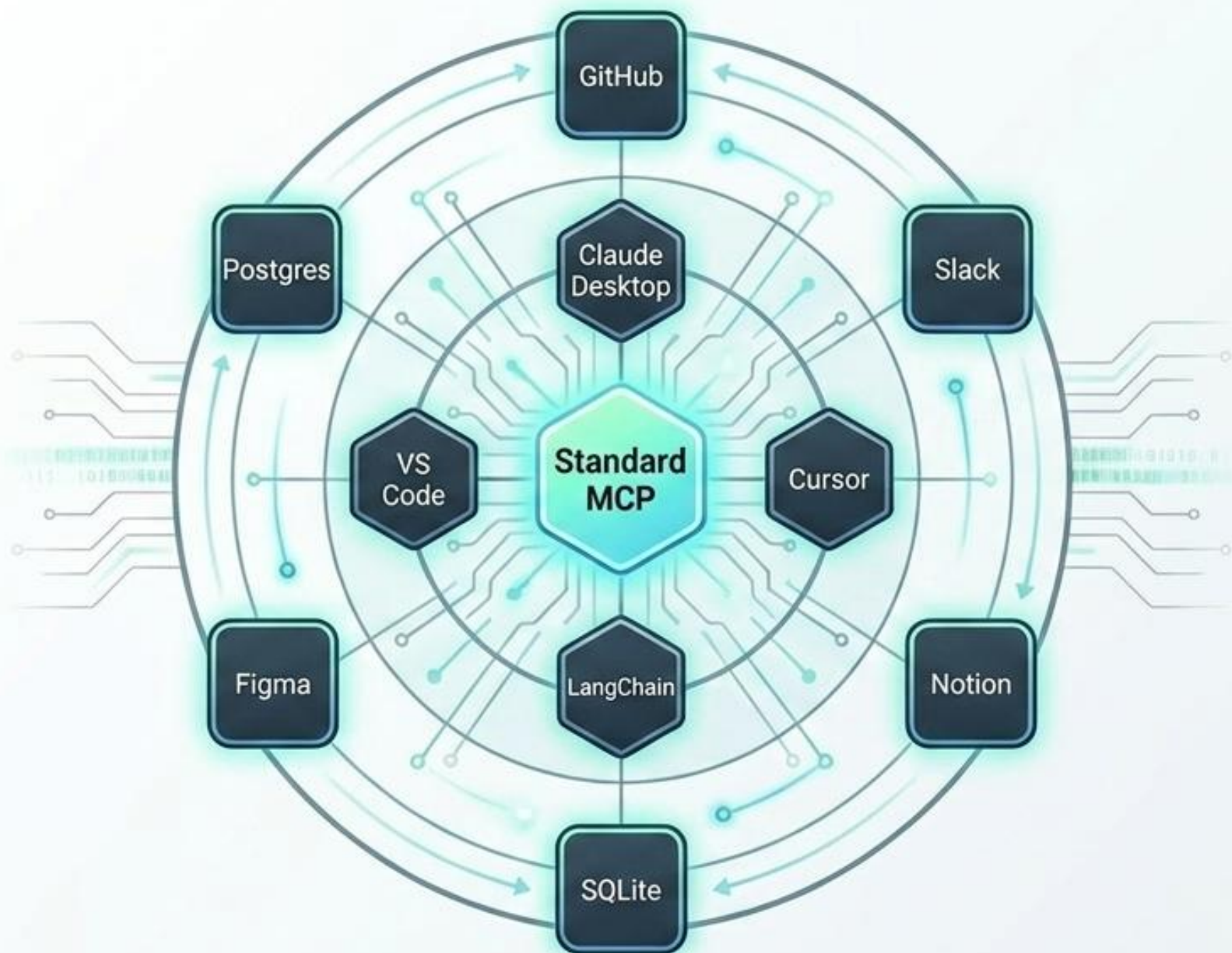
**Serveurs MCP Virtuels :** Ne pas exposer l'outil brut. Créer une couche abstraite. Exemple : Exposer `create_github_issue`, mais masquer et interdire `delete_repository`. Le Gateway gère cette curation dynamiquement.



**Human-in-the-Loop (HitL) :** Le principe du moindre privilège. Pour toute action destructrice (ex: `DROP TABLE`, `SEND EMAIL`), le Gateway suspend l'appel MCP et exige une validation explicite de l'utilisateur humain.



# L'Écosystème MCP en 2026 : L'Adoption Massive



## Un Standard Ouvert :

Hébergé par la Linux Foundation, soutenu par Anthropic, Google DeepMind, et OpenAI.

## Effet Réseau :

L'écosystème évolue de semaine en semaine. La valeur d'un serveur MCP augmente avec chaque nouveau client IA compatible.

## La Stratégie :

Ne vous enfermez plus dans des intégrations spécifiques à un fournisseur. Pariez sur le standard ouvert.